

Modelos de linguagem locais

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

12 de junho de 2026



- **Ollama** é uma plataforma para executar LLMs localmente.
- Simplifica download, versionamento e execução de modelos.
- Fornece API HTTP compatível com aplicações externas.
- Foco em privacidade, controle e baixa latência.



- Privacidade de dados sensíveis.
- Redução de custos com APIs externas.
- Baixa latência.
- Customização e fine-tuning local.
- Uso offline.



- Cliente (CLI ou aplicação)
- Servidor Ollama (API REST)
- Modelo baixado (ex: llama3, mistral)
- Execução via CPU ou GPU



- Docker instalado
- Mínimo recomendado:
 - 16GB RAM
 - CPU moderna ou GPU compatível
 - Espaço em disco para modelos (4GB+)

Baixar, criar o container e executar o serviço ollama.

```
docker run -d -p 11434:11434 ollama/ollama:latest --name ollama
```

Parar ou iniciar (depois de criado)

```
docker start ollama  
docker stop ollama
```

Baixar um modelo de linguagem. Lista de mmodelos.

- LiquidAI/lfm2.5-1.2b-instruct
- gemma4

```
docker exec -it ollama ollama pull LiquidAI/lfm2.5-1.2b-instruct
```

Iniciar o servidor de modelos de linguagem.

```
docker exec -it ollama ollama
```

Após iniciar o container:

```
docker exec -it ollama ollama pull llama3.1
```

Ou via API:

```
curl http://localhost:11434/api/pull \  
-d '{"name": "llama3"}'
```

1. Baixar a imagem oficial:

```
docker pull ollama/ollama
```

2. Executar o container:

```
docker run -d \  
-v ollama:/root/.ollama \  
-p 11434:11434 \  
--name ollama \  
ollama/ollama
```



Via CLI:

```
docker exec -it ollama ollama run llama3
```

Via API:

```
curl http://localhost:11434/api/generate \  
-d '{  
  "model": "llama3",  
  "prompt": "Explique LLMs"  
}'
```




- API REST disponível em:
`http://localhost:11434`
- Pode ser integrada com:
 - Python
 - Node.js
 - Sistemas corporativos



- Listar modelos:

```
ollama list
```

- Remover modelo:

```
ollama rm llama3
```

- Versionamento automático



- Monitorar uso de memória e CPU/GPU
- Usar volumes Docker persistentes
- Avaliar quantização de modelos
- Restringir acesso externo à API
- Testar modelos menores antes de escalar

Arquitetura:

- Navegador → Open WebUI
- Open WebUI → API HTTP do Ollama
- Ollama → Modelo local (ex: llama3)

Executando Open WebUI com Docker:

```
docker run -d \  
-p 3000:8080 \  
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
--name openwebui \  
ghcr.io/open-webui/open-webui:main
```

Acesse:

<http://localhost:3000>